Neighborhood Services — Technical Documentation

> A full-stack home-services marketplace for the Egyptian market, connecting customers with local technicians across three booking models, with an AI layer for moderation, matchmaking, visual problem analysis, pricing guidance, and a customer chatbot.

Stack at a glance: .NET 10 · Clean Architecture · MediatR (CQRS) · EF Core 10 (SQL Server) · Hangfire · Redis · Semantic Kernel + Qdrant (RAG) · SignalR · Angular 21

Table of Contents

---

1. Project Overview

Neighborhood Services is a two-sided marketplace web application that links customers who need home maintenance and repair work (plumbing, electrical, appliances, cleaning, etc.) with verified local technicians. It is built as an ITI graduation project by a team of seven, with a .NET 10 backend following Clean Architecture and an Angular 21 single-page frontend.

The platform supports three distinct ways to get work done — **direct booking**, **competitive bidding**, and **recurring scheduled service** — and layers an **AI subsystem** on top for content moderation, technician matchmaking, image-based problem analysis, price guidance, review/dispute analysis, and a knowledge-grounded customer chatbot.

---

2. Problem Statement & Objectives

Problem. Finding a trustworthy technician for a home problem is slow and opaque: customers don't know who is qualified, who is nearby, what a fair price is, or whether a provider is reliable. Providers, in turn, lack a structured channel to receive jobs, quote prices, and build a reputation.

Objectives.

- Provide a single platform where customers can describe a problem (in Arabic or English, with a photo) and reach the right technician quickly.
- Offer flexible engagement models: book a specific technician directly, post a request and accept competing offers, or set up a repeating schedule.
- Use AI to reduce friction and increase trust — auto-moderate user content, recommend the best technician for a problem, estimate prices, analyze an uploaded photo, and answer questions via a chatbot.
- Handle the full commercial loop: wallets, escrow, promo codes, invoices, reviews, and disputes.
- Be bilingual (Arabic / English with RTL) and production-shaped (caching, background jobs, notifications, structured logging of every AI decision).

---

3. What the System Does

Three marketplace flows (all closed two-sided)

Flow	Customer side	Technician side
Direct booking	Picks a technician, books a service, confirms completion	Accepts job (sets duration), marks complete
Competitive bidding	Posts a service request, reviews incoming offers, accepts one	Browses nearby open requests (geo radius), makes an offer
Recurring service	Sets up a recurring schedule, approves the quoted price	Sets the recurring price; the system auto-generates bookings

Supporting capabilities

- **Identity & profiles** — registration/login (local + Google OAuth), customer addresses, technician profiles, photos/portfolio, categories, availability, and per-service pricing.

- **Money** — wallet top-up (Paymob/PayPal), escrow on bookings, promo codes applied at accept time, invoices (PDF), and technician payouts/earnings.
- **Trust & support** — reviews (AI-analyzed), disputes (raise/resolve), support tickets, and a staff oversight back office.
- **Communication** — real-time customer↔technician chat (SignalR), in-app notifications with a topbar bell, email, and newsletter.
- **AI** — moderation, matchmaking, image analysis, price guidance, review/dispute analysis, and a RAG-grounded **tool-using chatbot** (text + image/vision) that can recommend technicians, check availability, estimate prices, and **place a booking** end-to-end in the chat.

---

4. Technology Stack

Backend

Concern	Technology
Runtime / language	.NET 10, C#
Architecture	Clean Architecture (4 projects) + CQRS
Mediator / CQRS	MediatR 14
ORM / DB	Entity Framework Core 10 · SQL Server
Geospatial	NetTopologySuite (distance / geo queries)
Identity & auth	ASP.NET Core Identity, JWT Bearer (httpOnly cookie), Google OAuth
Background jobs	Hangfire 1.8 (SQL Server storage)
Caching	Redis (StackExchange.Redis)
AI orchestration	Microsoft Semantic Kernel 1.77 (OpenAI chat + vision + embeddings)
Vector store (RAG)	Qdrant 1.18
Real-time	SignalR
Object mapping	Mapster
PDF generation	QuestPDF
Media storage	Cloudinary
Email	MailKit / MimeKit
API docs	Swagger / Swashbuckle

Frontend

Concern	Technology
Framework	Angular 21 (standalone components, feature-based structure)
UI	Bootstrap 5, ng-bootstrap, Bootstrap Icons, FontAwesome
i18n	ngx-translate (Arabic / English, full RTL)
Real-time	@microsoft/signalr
Charts	chart.js
Alerts / toasts	SweetAlert2, ngx-toastr
Carousel	ngx-owl-carousel-o

---

5. System Architecture

The backend follows **Clean Architecture** with four projects and a strict dependency rule (dependencies point inward; the Domain has no outward dependencies):

```
Neighborhood.Services.API           → Controllers, DI composition root, Program.cs, Hangfire dashboard
Neighborhood.Services.Application   → CQRS (Commands/Queries + Handlers via MediatR), DTOs, interfaces, business rules
Neighborhood.Services.Domain        → Entities, enums, domain exceptions (pure, no framework deps)
Neighborhood.Services.Infrastructure → EF Core, repositories, external services (AI, payments, email, Cloudinary, Hangfire)
```

Key patterns

- **CQRS via MediatR.** Every use case is a `Command` or `Query` with a dedicated handler. This keeps controllers thin (they just `Send` a request) and makes each operation independently testable. Example: `SendChatMessageCommand` → `SendChatMessageCommandHandler`.
- **Repository + Unit of Work.** Data access is behind repository interfaces in Application, implemented in Infrastructure; `IUnitOfWork.SaveChangesAsync` commits transactions.
- **Seams over direct dependencies.** The Application layer never references Hangfire, Qdrant, or OpenAI directly — it talks to interfaces (`IBackgroundJobScheduler`, `IVectorMemory`, `IAiClient`, `IKnowledgeIndexer`) that Infrastructure implements. This keeps the core portable and the AI/infrastructure swappable.
- **Fail-open AI.** Every AI call is wrapped so that an outage (bad key, quota, Qdrant down) never breaks the user-facing flow — the request proceeds without the AI enhancement.

Request lifecycle (example: post a service request)

```
Controller → MediatR Command → Handler (validate, persist) → SaveChanges
   → enqueue Hangfire moderation job (returns immediately)
   → [worker] AI moderation → set Open/Flagged → notify customer/staff
```

---

6. Domain Model & Data

Entities are grouped by bounded context / owner. (~44 entity types, EF Core code-first with migrations.)

Context	Entities
Booking / Marketplace	`SERVICE_REQUEST`, `OFFER`, `BOOKING`, `RECURRING_BOOKING`, `BOOKING_IMAGE`, `CANCELLATION_POLICY`, `AI_ANALYSIS`, `AGENT_LOG`
Identity	`USER`, `CUSTOMER`, `TECHNICIAN`, `CUSTOMER_ADDRESS`, `TECHNICIAN_PHOTO`
Payments	`WALLET`, `TRANSACTION`, `ESCROW`, `PAYMENT_METHOD`, `INVOICE`, `PROMO_CODE`, `PROMO_CODE_USAGE`
Support & Trust	`STAFF`, `STAFF_PERMISSION`, `SUPPORT_TICKET`, `SUPPORT_MESSAGE`, `DISPUTE`, `REVIEW`, `REVIEW_ANALYSIS`
Communication	`CONVERSATION`, `MESSAGE`, `NOTIFICATION`, `NEWSLETTER`, `FAVORITE`
Catalog & Config	`CATEGORY`, `PROBLEM_TYPE`, `TECHNICIAN_CATEGORY`, `TECHNICIAN_PRICING`, `TECHNICIAN_AVAILABILITY`, `AVAILABILITY_EXCEPTION`, `HISTORICAL_PRICES`
Chatbot	`CHATBOT_SESSION`, `CHATBOT_MESSAGE`

Notable design points:

- **Bilingual catalog.** `CATEGORY` and `PROBLEM_TYPE` carry `NameEn` / `NameAr`; the same problem type is embedded into the vector store so the chatbot/classifier works in both languages.
- **Pricing data, not in RAG.** `HISTORICAL_PRICES` lives in SQL and drives the price-estimation service; only catalog text is embedded in Qdrant.
- **AI is auditable.** Every AI decision writes an `AGENT_LOG` row (agent type, action, input, output, token usage, and the entity it refers to). `AI_ANALYSIS` stores structured image-analysis results.

---

7. Functional Modules

Each module is a vertical slice — domain entities, CQRS commands/queries, repository, controller, and the matching Angular feature. (See §12 for which team member owns each module.)

Identity & Authentication

Controllers: `AuthController`, `UsersController`.

The authentication backbone every other module depends on. Local login/registration and **Google OAuth** (`google-login` / `google-callback`) are issued as a JWT in an httpOnly cookie on top of ASP.NET Core Identity. The user lifecycle is covered end-to-end: profile and location updates, activate/deactivate, role queries, and a `nearby` user lookup.

Technician & Customer Profiles

Controllers: `TechniciansController`, `TechnicianPhotosController`, `CustomerAddressesController`, `GeocodingController`.

Rich technician profiles — public profile cards, a verification-status workflow (`PATCH verification-status`), availability toggles, browse listing, photo/portfolio management, and **available/busy time-slot** computation that feeds the booking flow. Customer addresses support defaults, soft-delete and restore. The `IGeocodingService` (forward `search` + `reverse` geocoding) underpins distance-based browse, matchmaking, and the chatbot's region resolution.

Marketplace Core (Bookings, Requests, Offers, Recurring)

Controllers: `BookingsController`, `ServiceRequestsController`, `OffersController`, `RecurringBookingsController`, `CancellationPoliciesController`.

The heart of the platform — the three marketplace flows, each closed two-sided:

- **Direct booking** — book → accept (set duration) → complete → confirm.
- **Competitive bidding** — post request → browse nearby requests → offer → accept.
- **Recurring service** — set up schedule → set price → approve → auto-generate occurrences.

It also covers cancellation policies and the paged "mine" listings on both the customer and technician sides.

AI & Knowledge

Controllers: `ChatbotController`, `AiAnalysisController`, `KnowledgeController`, `AgentLogsController`.

The customer-facing AI surface: the RAG-grounded **chatbot**, image-based **problem analysis**, the **knowledge (RAG) reindex** endpoint, and the **AgentLog** audit trail that records every AI decision. The agents themselves and the foundation behind them are detailed in §8.

Background Processing

Components: `IBackgroundJobScheduler`, `BackgroundJobScheduler`, `RecurringBookingGeneratorService`, `ServiceRequestExpiryService`, `ServiceRequestModerationJob`, `KnowledgeIndexJob`.

A **Hangfire**-backed background-job layer (SQL Server storage, dashboard at `/hangfire`) that pushes slow or failure-prone work off the request path. It runs the daily **recurring-booking generator** and **request-expiry** sweeps, and handles fire-and-forget jobs for **AI moderation** and **RAG index synchronization**. The Application layer enqueues through a thin `IBackgroundJobScheduler` seam so it never depends on Hangfire directly. Full job catalogue in §9.

Payments & Wallets

Controllers: `WalletsController`, `TransactionsController`, `PaymentsController`, `PaymentMethodsController`.

The money rails. The **Paymob** gateway is integrated through a clean three-step server flow (`PaymentGatewayService`: authenticate → create order → request payment key) with a `paymob/callback` webhook to confirm and finalize wallet top-ups; PayPal is also supported. Wallets support top-up, withdrawal, and payment verification/finalization; transactions are typed and queryable per wallet.

Escrow, Promo Codes & Invoicing

Controllers: `EscrowsController`, `PromoCodesController`, `InvoicesController`.

Escrow holds a booking's funds and exposes explicit `release` / `refund` operations — the mechanism that makes the marketplace trustworthy, since the customer's money is held until the job is done. **Promo codes** offer validate / preview / apply semantics and are applied at accept-time against the final price. **Invoices** are generated as polished **PDFs via QuestPDF**, retrievable per booking / customer / technician, with inline-view and download endpoints and a void operation.

Service Catalog & Technician Configuration

Controllers: `CategoriesController`, `ProblemTypesController`, `TechnicianCategoryController`, `TechnicianPricingController`, `TechnitianAvailabilityController`, `AvilabilityExceptionController`, `HistoricalPricesController`.

The **service catalog** the whole marketplace is built on: bilingual categories and problem types (each carrying a min/max price band), the technician↔category mapping, per-service technician pricing, weekly availability plus one-off availability **exceptions**, and the **historical-prices** dataset that grounds price estimation.

Caching

Components: `IResponseCacheService`, `ResponseCacheService`, `[Cache(ttl)]` action filter.

An application-wide caching layer — a Redis-backed cache service plus an action filter that transparently caches read-heavy endpoints (categories, problem types) keyed by request path + query string, and degrades gracefully when Redis is unavailable.

Trust & Safety (Reviews, Disputes, Support)

Controllers: `ReviewsController`, `ReviewAnalysisController`, `DisputesController`, `SupportTicketsController`, `SupportMessagesController`.

Reviews support reviewer / reviewee / status / flagged queries and are surfaced alongside their AI **review-analysis** records. **Disputes** are filterable by status / type / user. **Support tickets** carry threaded messages (nested `…/messages` route, read-receipts, priority and status workflow). Together these power the staff oversight pages: bookings oversight, the flagged-request queue, dispute resolution, and review moderation.

Staff & File Service

Controllers: `StaffController`, `FilesController`.

Staff management with role-based permissions. The **File / Image service** is a Cloudinary-backed upload-signature endpoint used across the app for secure direct-to-cloud uploads.

Communication (Chat & Notifications)

Controllers: `ConversationController`, `MessagesController`, `NotificationsController`, plus the SignalR `ChatHub`.

Real-time **conversations + messages** power customer↔technician chat over a SignalR `ChatHub` (private, group and broadcast sends, join/leave groups, connect/disconnect tracking), with messages also queryable per booking. The **notification service** delivers in-app notifications (per-user, directive, mark-as-read, mark-all-read) through a dedicated `NotificationHub` feeding the topbar bell.

Engagement (Email, Newsletter, Favorites)

Controllers: `EmailController`, `NewsletterController`, `FavoritesController`.

The outbound-engagement layer: an **email service** (MailKit/MimeKit with templated content), a **newsletter** signup + send, and **favorites** so customers can save technicians.

---

8. The AI Subsystem

The AI layer is one of the project's most distinctive features: a shared foundation over which a set of focused agents each handle one job. The sections below describe each part by capability.

8.1 Foundation

A thin, provider-agnostic abstraction (`IAiClient`) over Microsoft **Semantic Kernel**, so every agent talks to one interface and the underlying model provider (OpenAI) is swappable.

`SemanticKernelClient` (Infrastructure) implements three methods:

- `CompleteAsync(systemPrompt, userPrompt, imageUrl?, log?)` — single-shot completion, with **vision support** (an image URL is sent as an `ImageContent` alongside the text).
- `ChatAsync(history, systemPrompt, log?)` — multi-turn completion driven by a `ChatHistory` .
- `ChatWithToolsAsync(history, systemPrompt, tools, log?)` — **tool-calling** completion. It clones the kernel per request, registers the supplied tool objects (`Plugins.AddFromObject`), and runs with `FunctionChoiceBehavior.Auto()` so the model can **decide to call tools on its own**, Semantic Kernel executes them, feeds the results back, and loops until a final answer. This is what powers the agentic chatbot (§8.3).

All methods extract **token usage** from the response metadata and, when an `AiCallContext` is supplied, persist an **AgentLog** entry (agent type, action, input, output, tokens, reference entity). Logging is wrapped in try/catch so an audit-write failure never breaks the AI call.

`AgentType` enum: `Matching` , `Pricing` , `Booking` , `QA` , `Moderation` , `Chatbot` .

8.2 RAG (Retrieval-Augmented Generation)

- **Vector store:** Qdrant, behind `IVectorMemory` (`UpsertAsync/UpsertManyAsync` , `SearchAsync` , `SearchDetailedAsync` returning `SearchHit(Text, Score, Fields)` , plus `RemoveAsync` / `RemoveExceptAsync` for pruning).
- **Indexer:** `KnowledgeSeeder` implements `IKnowledgeIndexer` . It reads the catalog **from the database**, embeds each category and problem type, and is:
- **Idempotent** — each Qdrant document stores a `_hash` ; only changed docs are re-embedded.
- **Batched** — embeddings sent via `UpsertManyAsync` .
- **Self-cleaning** — `RemoveExceptAsync` prunes vectors whose source rows no longer exist.
- **Off the boot path** — rebuilt deliberately via CLI (`-- reindex-knowledge`) or `POST /api/knowledge/reindex` (staff-only).
- **Event-driven sync:** catalog CRUD enqueues Hangfire jobs that keep the index live (see §9).

Two collections are maintained: `platform-knowledge` (general Q&A grounding) and `problem-types` (each embedded **with its** `problemTypeId` **payload**, enabling free-text → problem-type classification).

8.3 The Agents

(1) Customer Chatbot — `SendChatMessageCommandHandler`

The flagship agent: a RAG-grounded, **tool-using** assistant. Rather than pre-computing answers in code, the chatbot is given a set of **callable tools** and the model itself **decides which to call (and chains them)** to walk a customer from a vague problem all the way to a placed booking — e.g. "*my kitchen tap is leaking, who's near me?*" → recommend → "*is Khaled free tomorrow?*" → check availability → "*how much?*" → estimate → "*book the 4 pm*" → create booking.

The tools (Semantic Kernel functions in `Application/Chatbot/Tools/`, run via `ChatWithToolsAsync` with auto function-calling):

Tool	What it does
`estimate_price(service, city?)`	Classifies the problem to a `problemTypeId` and returns a grounded, region-localized estimate from the **price-estimation service**.
`recommend_technician(problem)`	Classifies the problem, then calls the existing **Matchmaking agent** (§8.3.4) to return the best-fit technicians by need.
`find_technicians(name, category?)`	Looks a technician up by name (tolerant, token-based matching), returning a short candidate list to disambiguate.
`check_availability(technicianId, date)`	Returns a technician's free start-times for a day (wraps the available-slots query).
`create_booking( … )`	The only **write** tool — places a Direct booking. **Customers only**, requires shared location, and uses a two-step **confirmation gate** (returns a summary first; only books on an explicit confirm). The booking is created **PENDING** — the technician still reviews and quotes; nothing is charged. It re-checks live availability at book time so a stale slot is never booked.

Conversation memory. Logged-in users have their context rebuilt from a persisted `ChatbotSession` (server-authoritative, capped to the recent window); guests get memory too via the frontend replaying the recent turns with each message. Tool results (e.g. the ids a tool returned) are persisted as `Tool`-role messages and replayed as context, so the agent "remembers" what it found across turns without re-searching.

Still RAG-grounded & multimodal. It injects top hits from `platform-knowledge`, accepts an attached **image** (vision) to describe the likely problem, resolves the user's region from GPS or text via the shared region resolver, and is wrapped by a hardened system prompt (strictly on-topic, prompt-injection resistant, never invents a city, replies in the user's language). Every tool and external step is independently wrapped so the bot degrades gracefully to a plain RAG answer.

(2) Image Analysis Agent — `AnalyzeBookingCommandHandler`

Vision agent that takes a problem description + photo and returns **structured JSON** (`detectedProblem` , `confidenceScore` , `estimatedMin/MaxPrice` , `severityLevel`). Results are persisted to `AI_ANALYSIS` and logged. Powers the customer chatbot's "what's wrong in this photo" and the technician's "Detect from photo" feature.

(3) Content Moderation Agent — `ModerateServiceRequestCommandHandler`

Runs on a Hangfire worker after a service request is posted. A vision-capable moderation prompt classifies the text + photo as appropriate/inappropriate (tuned **not** to flag normal messy repair photos). Sets the request to `Open` or `Flagged` , then notifies the customer or staff. **Fails open** — any AI failure lets the post go live. **Idempotent** — safe if Hangfire re-runs.

(4) Matchmaking Agent — `GetTechnicianMatchesQueryHandler`

A hybrid **rules → LLM → rules-fallback** recommender powering the "Smart Match" feature:

1. **Rules filter** — hard-filters technicians to the right category and within travel range, scores them by a weighted composite (rating 40%, distance 30% via Haversine, verification 20%, availability 10%), and shortlists the top 15.
2. **LLM rank** — the shortlist + the customer's own problem text are handed to the model, which re-ranks best-first and writes a one-line reason per technician (language-aware).
3. **Fallback** — if the LLM fails or returns nothing usable, it falls back to the rule order and records *why* in `AgentLog` . The response flags `RankedByAi` so the UI knows which ranking it got.

(5) QA Agent — `QaAgent`

Analyzes **reviews** (sentiment, 0–100 quality score, abuse flag → persisted as `REVIEW_ANALYSIS` , and auto-flips the review's status to Approved/Flagged) and **disputes** (severity + whether human review is required). Returns structured JSON; logs to `AgentLog` .

(6) Price-Estimation Agent — `PriceEstimationService`

Produces a single grounded estimate for a problem type and optional region. It averages `HISTORICAL_PRICES` (region-specific if available, else overall); when no history exists it falls back to the problem type's `[MinPrice, MaxPrice]` midpoint, scaled by a per-region multiplier (Cairo 1.3× … Mahalla 1.0×). Rule-based and deterministic — consumed by both the "Get Estimate" button and the chatbot's authoritative-pricing path.

8.4 Auditing — AgentLog

Every agent decision is recorded via `CreateAgentLogCommand` with agent type, action, input, output, token usage, and the referenced entity. The **chatbot** logs one row per turn (input = user message, output = final reply, + tokens) under `AgentType.Chatbot` , and when it **places a booking** it writes an

additional `CreateBooking` audit row that references the real booking (`ReferenceType=Booking`) so the action is traceable to the entity it created.

Logs are surfaced two ways through `AgentLogsController` :

- **By reference** — `GET /agentlogs/reference/{type}/{id}` returns all agent activity on one entity (e.g. every AI touch on Booking #42).
- **Admin viewer** — a full-access staff page (`/staff/agent-logs`) with **tabs per agent type**, full-text **search** over input/output, **pagination**, and a **row-details modal** that shows the complete input/output (scrollable). Backed by a paged, filtered query (`GET /agentlogs?type=&search=&from=&to=&page=&pageSize=` , gated by `[HasPermission(FullAccess)]`).

This makes the AI layer **observable and explainable** — a key requirement for a trust-centric marketplace.

---

9. Background Processing (Hangfire)

Background work runs on **Hangfire** with **SQL Server storage** and a dashboard at `/hangfire` . The Application layer enqueues work through a thin seam (`IBackgroundJobScheduler`) so it never depends on Hangfire directly.

Recurring jobs (registered in `Program.cs`)

Job	Schedule	Purpose
`RecurringBookingGeneratorService.GenerateBookings`	`Cron.Daily`	Looks ahead 7 days and generates concrete bookings for every due recurring schedule (daily/weekly/monthly pattern matching).
`ServiceRequestExpiryService.ExpireOpenRequestAndOffer`	`Cron.Daily`	Expires stale open service requests and their offers.

Fire-and-forget jobs (enqueued on demand via `BackgroundJobScheduler`)

Job	Trigger	Purpose
`ServiceRequestModerationJob.Run`	After a service request is created	Runs the AI moderation agent off the request thread.
`KnowledgeIndexJob.IndexCategory` / `RemoveCategory`	Category CRUD	Re-embeds (or removes) a category **and cascades to its problem types** (the category name is embedded in each child).
`KnowledgeIndexJob.IndexProblemType` / `RemoveProblemType`	Problem-type CRUD	Re-embeds (or removes) a single problem type's vectors.

`KnowledgeIndexJob` caps retries at **3 attempts over 1 / 5 / 15 minutes** — if embedding fails because of quota/Qdrant downtime, retrying the default 10× just burns a worker; the row stays in the DB and a later `/reindex` re-syncs it. All index jobs are enqueued **after** the DB write commits, so a slow AI call never blocks (or fails) the admin's CRUD request.

Why this matters: the heavy/slow/failure-prone work (LLM moderation, embeddings, recurring generation) is pushed off the request path, keeping the API responsive and resilient.

---

10. Frontend (Angular)

Angular 21, standalone components, organized by **feature** with role-based layouts. The app is fully bilingual (Arabic/English) with RTL support via ngx-translate.

```
src/app/
  core/        guards, interceptors (auth, language, error), resolvers, services,
models
  layouts/     public · auth · customer · technician · staff · dashboard-shell
  features/
    public/    home, services (catalog), about, contact, customer-support
    auth/      login, register (customer/technician), forgot-password, OAuth callback
    customer/  bookings, service-requests, recurring-bookings, find-technician,
wallet,
               favorites, chats, notifications, profile, dashboard
    technician/ jobs, browse-requests, offers, recurring-jobs, availabilities,
pricing,
               categories, gallery, earnings, wallet, reviews, chats, profile,
dashboard
    staff/     dashboard, bookings (oversight), categories, problem types, disputes,
               flagged-requests, reviews (+ analysis), support-tickets, promo-codes,
               policies, users, staff-management, newsletter
  shared/      reusable components (e.g. favorite button), pipes, utilities
```

Cross-cutting frontend concerns: a cookie-based auth interceptor (`withCredentials`), a language interceptor, a centralized error interceptor, a reactive navbar, route guards (auth + guest), SignalR clients for chat and the notification bell, and a shared `ApiService` that prefixes `/api` .

---

11. Cross-Cutting Concerns

- **Authentication.** Cookie-based JWT (`access_token` , httpOnly, `SameSite=None; Secure`) so the Angular SPA (`localhost:4200`) can call the API (`localhost:7228`) cross-site. Google OAuth supported. ASP.NET Core Identity backs user management. The same cross-site cookie setup applies in production between the deployed frontend and API (see §14); because it is a third-party cookie across those domains, strict browser privacy settings can block it.
- **Caching.** Redis via a `[Cache]` attribute on read-heavy endpoints (e.g. categories), with a response-cache service.
- **Notifications.** Domain-wide in-app notifications (SignalR) with a topbar bell, plus email and newsletter. Notification sends are best-effort (wrapped in try/catch) so they never undo a business action.
- **Internationalization.** Full Arabic/English with RTL across all dashboards; the catalog and AI responses are bilingual.
- **Geospatial.** NetTopologySuite + Haversine for distance-based browse/matchmaking; browser geolocation on the customer side.
- **Observability.** Structured logging throughout, plus the AgentLog audit trail for AI.
- **Resilience.** Fail-open AI, idempotent background jobs, capped retries, and graceful degradation when Redis/Qdrant/OpenAI are unavailable.

---

12. Team Contributions

Member	Service ownership	Controllers / domain	Frontend area
Ahmed	Background-job service (Hangfire), AI foundation & agents	Bookings, ServiceRequests, Offers, RecurringBookings, CancellationPolicies, AgentLogs, Chatbot, AiAnalysis, Knowledge	Customer marketplace flows, technician job flows, AI Analysis / Agent Log + chatbot UI
Mahmoud	Authentication + Google OAuth, Geocoding service (`IGeocodingService`)	Auth, Users, Technicians, CustomerAddresses, TechnicianPhotos, Geocoding	Auth pages, customer & technician profiles, technician public profile card
Ziad	Payment gateway (Paymob/PayPal) + escrow, PDF/invoice service	Wallets, Transactions, Payments, PaymentMethods, Invoices, PromoCodes, Escrows	Customer & technician wallets, earnings/invoices, checkout, staff promo codes
Alaa	Caching service, QA agent + price-estimation agent	Categories, ProblemTypes, TechnicianPricing, TechnicianAvailability, AvailabilityExceptions, TechnicianCategories, HistoricalPrices	Staff catalog & policies, technician availability & pricing, public services catalog
Amira	File / image service (Cloudinary)	Staff, SupportTickets, SupportMessages, Disputes, Reviews, ReviewAnalysis, Files	Staff back office: staff-management, support tickets, disputes, reviews & analysis, bookings oversight; leave-a-review UI
Arwa	Email service, notification service (SignalR)	Conversation, Messages, Notifications, Favorites, Newsletter, ChatHub	Customer/technician chat, notifications + bell, favorites, newsletter, public home/support

13. Setup, Run & Configuration

13.1 Prerequisites

Tool	Version / notes
.NET SDK	**10.0**
Node.js	LTS (18+)
Angular CLI	**21.x** (`npm i -g @angular/cli`)
SQL Server	any recent edition (Express works) — hosts the app DB **and** Hangfire storage
Redis	required for `[Cache]` endpoints (locally on Windows, run via WSL; production uses a managed **Redis Cloud** instance — see §14)
Qdrant	optional — only needed for the RAG chatbot / knowledge search
EF Core tools	`dotnet tool install --global dotnet-ef`

External service keys are read from configuration. The committed `appsettings.json` already contains the SQL connection string, Redis, Cloudinary and email settings; the **secret** keys are supplied via .NET **user-secrets** (the API project has a `UserSecretsId`):

```
# from Neighborhood.Services.API
dotnet user-secrets set "OpenAI:ApiKey" "<key>"
dotnet user-secrets set "Qdrant: … " "< … >"
dotnet user-secrets set "Authentication:Google:ClientId" "< … >"
dotnet user-secrets set "Authentication:Google:ClientSecret" "< … >"
dotnet user-secrets set "PaymentGateway:Paymob … " "< … >"
```

> The app boots even without OpenAI/Qdrant (dummy fallbacks) — AI features simply fail per-call until the keys are present.

13.2 Run the backend

```
# from Provider/Neighborhood-Services
dotnet ef database update -p Neighborhood.Services.Infrastructure -s
Neighborhood.Services.API
dotnet run --project Neighborhood.Services.API
```

- API runs at **https://localhost:7228** (HTTP **5001**). Swagger UI is served in Development.
- **Hangfire dashboard:** `/hangfire`. Recurring jobs register on startup.

- Make sure **Redis** is running first (Windows/WSL: `sudo service redis-server start`, verify with `redis-cli ping` → `PONG`), otherwise `[Cache]` endpoints fail.

Build the RAG knowledge index once (deliberate — it is **off** the boot path):

```
dotnet run --project Neighborhood.Services.API -- reindex-knowledge
# equivalent to POST /api/knowledge/reindex (staff-only)
```

13.3 Run the frontend

```
cd Client/neighborhood.services.client
npm install
ng serve        # http://localhost:4200  →  API https://localhost:7228
```

> The SPA calls the API cross-site with credentials (cookie auth), so both must be running and the API's CORS policy must allow `http://localhost:4200`.

13.4 Seed data

The DB seeder runs automatically **on an empty database**, creating sample customers / technicians (password `Pass@123`), the service catalog, and demo marketplace data. To re-seed, drop and recreate the database:

```
dotnet ef database drop -f -p Neighborhood.Services.Infrastructure -s
Neighborhood.Services.API
# then run the API again
```

Browse-requests use **real browser geolocation**; seeded requests are in Alexandria (lat 31.2001, lng 29.9187) — set DevTools → Sensors → Location to those coordinates to test.

---

14. Deployment (Production)

The application runs as a fully managed, **zero-cost** cloud stack. Frontend and backend are hosted separately and talk over HTTPS; every stateful service is a managed cloud offering.

14.1 Hosting topology

Component	Host	Notes
Frontend (Angular SPA)	**Vercel**	Static build, deployed via the Vercel CLI (`vercel --prod`). SPA routing via `vercel.json` rewrites.
Backend API	**MonsterASP.NET**	.NET 10 on Windows/IIS, deployed via Visual Studio **Web Deploy** (publish profile). HTTPS via free Let's Encrypt + force-redirect.
SQL Server + **Hangfire**	**MonsterASP.NET MSSQL**	One database serves both EF Core and Hangfire (`UseSqlServerStorage`). The app **auto-migrates and seeds on first boot** — no manual migration step.
Cache (Redis)	**Redis Cloud** (managed, free tier)	Backs the `[Cache]` response cache. The connection string carries `abortConnect=False` so a transient cache outage never blocks startup. Replaces the local WSL Redis used in development.
Vector store (RAG)	**Qdrant Cloud**	Backs the chatbot / knowledge search.
Media storage	**Cloudinary**	Direct-to-cloud uploads, so the host needs no writable disk.

Live URLs - Frontend — `https://neighborhood-services.vercel.app` - API — `https://neighborhoodservices.runasp.net`

14.2 Configuration — environment variables, not files

Production secrets are **never committed**. They are supplied as **host environment variables**, which ASP.NET Core's configuration layer reads and which **override** the values in `appsettings.json` . Nested keys use a **double-underscore (`__`)** separator (ASP.NET Core maps `Section__Key` → `Section:Key`). Locally these same keys come from .NET **user-secrets** (§13.1) — *same code, different source per environment.*

The keys the deployed API expects (names only — values live in the host's config panel):

```
ASPNETCORE_ENVIRONMENT
ConnectionStrings__DefaultConnection                    # cloud SQL Server
ConnectionStrings__Redis                                # Redis Cloud:
host:port,password=…,abortConnect=False
Jwt__Key   Jwt__Issuer   Jwt__Audience   Jwt__DurationInMinutes
Authentication__Google__ClientId   Authentication__Google__ClientSecret
OpenAI__ApiKey
Qdrant__ApiKey   Qdrant__Endpoint
Geoapify__ApiKey   Geoapify__BaseUrl
PaymentGateway__PaymobApiKey        PaymentGateway__PaymobIntegrationId
PaymentGateway__PaymobIframeId      PaymentGateway__PaymobHmacSecret
PaymentGateway__PaymobBaseUrl
Cloudinary__CloudName   Cloudinary__ApiKey   Cloudinary__ApiSecret
EmailSettings__Username   EmailSettings__Password   EmailSettings__SmtpServer
Cors__AllowedOrigins__0                                 # the deployed frontend
```

```
origin
AppUrl                                                  # public API base URL
```

14.3 Frontend build & deploy

The Angular app is environment-aware: on a production build, `environment.ts` (dev → `localhost`) is swapped for `environment.prod.ts` (production → the live API) via Angular's `fileReplacements`. The API base URL is **baked in at build time** (it is not a secret).

```
cd Client/neighborhood.services.client
npm run build          # production build (uses environment.prod.ts)
vercel --prod          # upload the build output (dist/.../browser) to Vercel
```

`vercel.json` defines the build command, output directory, and the SPA rewrite (`/* → /index.html`).

14.4 Backend deploy

In Visual Studio: right-click `Neighborhood.Services.API` → **Publish** → the MonsterASP.NET **publish profile** (Web Deploy). Re-publish to ship changes. On first boot the app creates the schema (its own tables + Hangfire's) and seeds demo data into the cloud database.

14.5 Operational notes

- **CORS.** The API allows exactly the deployed frontend origin via `Cors__AllowedOrigins__0`; the local default `http://localhost:4200` stays in `appsettings.json` for development.
- **Auth cookie.** `access_token` is `SameSite=None; Secure`. Across the Vercel↔Monster domains it is a third-party cookie; most browsers allow it, but strict privacy settings (Safari, hardened Brave) can block it. A custom domain (making both same-site) or header-based tokens removes the risk.
- **Hangfire dashboard.** `/hangfire` uses Hangfire's default **local-requests-only** authorization, so it is not publicly reachable in production (remote requests get `401`). Scheduled jobs still run on the server regardless.
- **Regional note.** `*.netlify.app` is blocked by some Egyptian ISPs, so the frontend is hosted on Vercel (`*.vercel.app`), which is reachable; `github.io` / `pages.dev` are alternative reachable hosts.

---

15. Future Work

- **Payment System Hardening** — robust concurrency hardening across all payment operations, plus a secure, user-friendly withdrawal/payout flow for technicians.
- **Live Technician Tracking** — real-time location & ETA on a map while a technician is on the way.
- **CI/CD Pipeline + Containerization (Docker)** — automated build/test/deploy and reproducible container images for the API and frontend.

- **Loyalty / Subscription Plans** — recurring-customer perks and subscription tiers.
